Classe de Teste Apex no Salesforce

Como construir uma classe de teste em apex quase perfeita no Salesforce

Guilherme Nunes Zwipp

Desenvolvedor Salesforce
https://www.linkedin.com/in/guilherme-zwipp-salesforce/

Público-alvo e Objetivo

Objetivo: Explicar o que são classes de teste em Salesforce (Apex), sua importância e como implementá-las de forma eficaz.

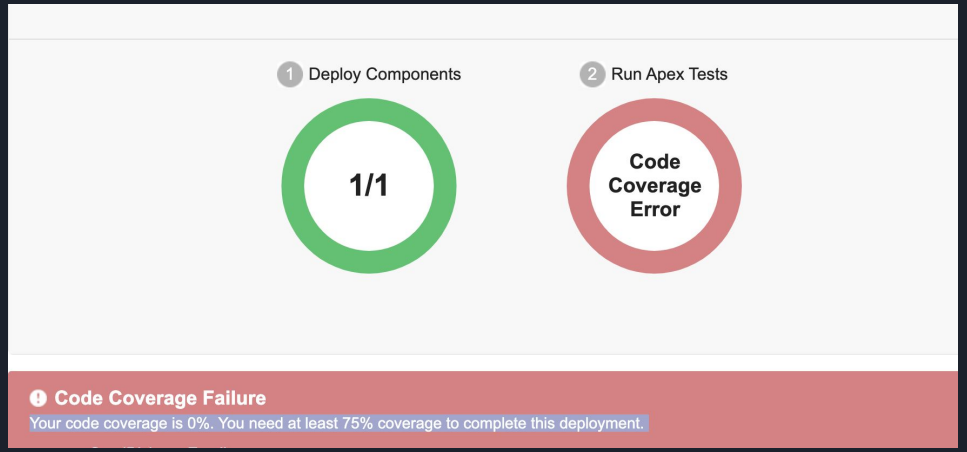
Público-alvo: Desenvolvedores com nível intermediário de experiência que desejam aprimorar a escrita de classes de teste.

O que são classes de teste?

Mais do que uma exigência para Deploy

Embora o **Salesforce** exija que pelo menos 75% do código esteja coberto por classes de teste para permitir o deploy, é importante entender que o papel das classes de teste vai muito além de apenas cumprir essa regra.

As classes de teste são fundamentais para validar se o código realmente cumpre o propósito para o qual foi desenvolvido, garantindo sua funcionalidade em diversos cenários, desde os mais amplos até os mais específicos. Elas são a prova real de que o sistema é robusto, confiável e preparado para lidar com situações inesperadas.



Qual é a sua importância?

Cobertura de Código

O Salesforce exige pelo menos 75% de cobertura de código para implantação em produção, mas conseguir esta cobertura não deve ser a prioridade!

Manutenção e Escalabilidade

Facilitar a manutenção do código ao longo do tempo, além de tornar o código mais fácil de ser alterado sem quebrar qualquer funcionalidade prévia.

Documentação comportamental

Forma de documentação. Sendo um tipo de consulta para visualizar o comportamento esperado em determinado cenário específico.

Qualidade e Confiabilidade

Identificar e corrigir erros antes que eles cheguem ao ambiente produtivo. Reduzindo acréscimos de tempo na entrega por conta de bugs inesperados.

Boas Práticas

F.I.R.S.T

Mencionado no livro *Clean Code* de Robert C. Martin, o acrônimo F.I.R.S.T. representa um conjunto de premissas para o desenvolvimento de testes de alta qualidade, promovendo maior confiabilidade, escalabilidade, usabilidade, e maior facilidade em manutenções futuras.

Fast

Os testes devem ser rápidos para garantir feedback ágil e um ciclo de desenvolvimento eficiente.

Independent

Cada teste deve ser executado de forma isolada, sem depender de outros. Isso evita falhas em cascata e facilita a identificação da origem de problemas.

Repeatable

Testes devem ser consistentes, independentemente do ambiente ou da ordem de execução.

Self-Validating

Cada teste deve indicar claramente sucesso ou falha por conta própria, sem a necessidade de inspeção manual dos resultados.

Timely

Os testes devem ser escritos de forma oportuna, idealmente pouco antes do desenvolvimento do código que será testado.

Boas Práticas

Responsabilidade Única

Cada método de teste deve validar apenas um único cenário de ponta a ponta. No entanto, se o código permitir a ocorrência de múltiplos cenários em uma única execução, essa combinação de cenários será considerada como um único cenário de teste.

Cenário reais

Escreva testes que reflitam situações reais, utilizando dados que sejam o mais próximo possível dos que serão usados em produção. Dessa forma, seu código estará melhor preparado para atender às demandas do mundo real.

Asserts Assertivos

Não adianta apenas verificar se o código executa sem erros; é essencial garantir que ele retorne exatamente o que deveria retornar. Para isso, utilize o máximo de asserts necessários para validar os resultados esperados em diferentes cenários.

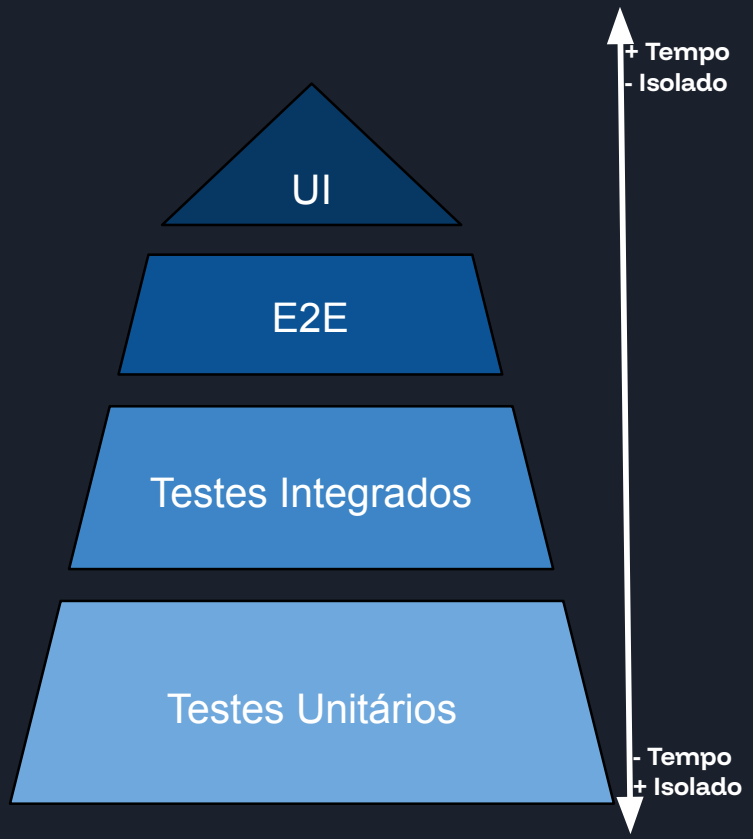
Testes Integrados x Testes Unitários

Tipos de Testes

Uma classe de teste não é apenas “uma classe de teste”. Atualmente, existem diferentes formas de se realizar testes, e cada uma delas é mais eficiente dentro de um determinado contexto.

Como esta apresentação trata especificamente de classes de teste em Apex, iremos nos concentrar em apenas dois tipos: testes integrados e testes unitários.

Inicialmente, abordaremos as diferenças entre esses dois tipos e quando utilizar cada um. Em seguida, veremos como construir uma classe de teste quase perfeita para cada abordagem.



Testes Integrados x Testes Unitários

Testes Integrados

- Verifica como diferentes componentes ou módulos de uma aplicação de software funcionam juntos, evitando potenciais bugs ao combinar diferentes partes do sistema.
- Neste tipo de teste, utilizamos dados "reais", realizando operações como inserção, modificação, exclusão ou consulta diretamente no banco de dados. Executando triggers, regras de validação, flows e outros fluxos associados ao objeto que está sendo utilizado para o teste.
- Exemplos de uso: TriggerHandler, Controller, Repository, Apex chamado no Flow etc...

X

Testes Unitários

- Verificam unidades individuais de código, como uma classe ou método, garantindo que estejam funcionando corretamente em todos os tipos diversos de cenários.
- Neste tipo de teste, não há interação com o banco de dados. Ele é realizado de forma isolada, focando exclusivamente na classe ou método que se deseja testar, garantindo que seu comportamento seja validado sem dependências externas.
- Exemplos de uso: Helper, Service, Enricher, Filter, TO/DTO etc...

Testes Integrados x Testes Unitários

Testes Integrados

X

Testes Unitários

Apenas o Começo...

Neste documento, apresentei apenas o início de algo maior que estou construindo.

Até aqui, exploramos a base teórica de como estruturar uma classe de teste quase perfeita em Apex.
Entendendo sua importância, boas práticas e seus diferentes tipos.

Mas teoria, por si só, não transforma código em excelência.

Na próxima parte deste documento, vamos para a prática: exercícios, exemplos reais e código (muito código).

Se você leu até aqui e o conteúdo te despertou curiosidade, fico no aguardo da sua presença na segunda parte.

Vamos evoluir isso juntos. 🚀